SessionRerank+: A Session-Aware Re-Ranking for API Tool Retrieval in Agentic Large Language Model Systems

Jenisha T[1] and Dr. Jyothi A P[2]

1 PhD Student, Department of Computer Science and Engineering, Faculty of Engineering and Technology, MS Ramaiah University of Applied Sciences, Bengaluru, India. Email: 24ETRP720001@msruas.ac.in

2 Associate Professor and Programme Head (M&C), Department of Computer Science and Engineering, Faculty of Engineering and Technology, MS Ramaiah University of Applied Sciences, Bengaluru, India. Email: jyothi.cs.et@msruas.ac.in

Abstract --- A Large Language Model (LLM) agent solves real-world tasks by calling external software services, called Application Programming Interfaces (APIs), over many conversational turns. Existing tool retrievers score each request as if the conversation had just begun, discarding strong cues already present in the session. We introduce SessionRerank+, a session-aware re-ranking layer that augments any dense retriever with three trajectory-derived affinity signals --- categorical co-occurrence, schema-signature similarity and endpoint pattern overlap --- read from a sparse Co-activation Cache with O(|H|) lookup. On all six ToolBench splits (1,100 queries, 3,211 APIs, 49 categories), SessionRerank+ raises mean session Normalised Discounted Cumulative Gain at rank 5 from 0.451 to 0.520 (+15.2%) and Hit at rank 5 from 0.726 to 0.791 (+8.9%), with $p < 5e\text{-}3$ on every split. By construction it reduces to the base retriever when the session is empty, so single-turn behaviour cannot regress.

Keywords --- Tool retrieval, agentic LLMs, session context, re-ranking, Co-activation Cache, ToolBench, multi-turn interaction, implicit personalisation, information retrieval.

1. INTRODUCTION

A conversational assistant no longer answers from its own knowledge alone. To accomplish concrete work --- book a flight, check a weather alert, query a land-records portal --- it must delegate the operation to an external software service exposed through an Application Programming Interface (API). Each API is a small, named contract: it states which arguments it accepts and what response it returns. The catalogue of available APIs is large; a modern agent sees thousands of them. Choosing the right one for each user request is therefore the central retrieval problem facing an agentic Large Language Model (LLM).

Two assumptions in the current generation of tool retrievers caused us concern. First, leading systems --- ToolLLM (Qin et al., 2024) and ToolRerank (Zheng et al., 2024) among them --- score every incoming request in isolation. They behave as if the user had just arrived, even when the conversation is several turns deep. Second, the standard remedy for "isolation" in classical Web search is user-level personalisation (Ai et al., 2018; Bennett et al., 2012), which depends on identified accounts, long search histories, and per-user training. Open-domain agent sessions have none of those things: they are typically anonymous and ephemeral, and operators are increasingly unwilling to store user-level traces.

The gap between these two positions is what this paper tries to close. Within a single session, a great deal of intent is already on the table: the APIs the agent has just called expose what family of task the user is pursuing, what kind of data they are manipulating, and what topical region of the corpus they are browsing. A human operator would exploit that trail; current tool retrievers do not. We treat the session's own trajectory as a free, anonymous, on-device training signal, and we show that a small layer reading three signals from it is enough to move the retrieval needle measurably.

Our contribution is SessionRerank+, a thin re-ranking layer that sits between any existing dense retriever and the agent. It does not retrain the retriever; it does not store user identities; and, when the session is empty, it collapses exactly to the base retriever it wraps. The remainder of this paper makes four innovations precise:

(i) A score-fusion formulation that augments the base relevance score with three trajectory-derived affinity signals --- categorical co-occurrence, schema-signature similarity and endpoint pattern overlap --- whose six combination weights are learned by Information Noise Contrastive Estimation (InfoNCE) without any user-level supervision.

(ii) The Co-activation Cache, a sparse symbolic memory of API-pair affinities updated by a simple increment-decay rule, giving O(|H|) lookup at serve time with no additional neural inference. The cache is a few megabytes of plain numbers; the entire layer runs on CPU.

(iii) An analytically provable cold-start guarantee: when the session is empty, all trajectory terms vanish and the score reduces exactly to the base retriever's. Deploying

SessionRerank+ therefore cannot regress single-turn behaviour --- a property that most personalised systems cannot claim.

(iv) A full evaluation on all six ToolBench splits showing a +15.2% relative gain in mean Normalised Discounted Cumulative Gain at rank 5 (NDCG@5) over the Dense baseline, and significant gains ($p < 5e-3$) on every split.

2. RELATED WORK

Tool retrieval for LLM agents was placed on a benchmark footing by ToolLLM (Qin et al., 2024), which constructed ToolBench from over 16,000 real-world RapidAPI endpoints, and extended by ToolRerank (Zheng et al., 2024), which adds an adaptive cross-encoder reranker. Both systems are stateless: they score the user request in isolation. The classical baselines on which they build share that limitation. BM25 (Robertson and Zaragoza, 2009) scores by lexical overlap; bi-encoder retrievers such as Sentence-BERT (Reimers and Gurevych, 2019) by embedding similarity; BERT-based cross-encoder re-rankers (Nogueira and Cho, 2019) by query-document interaction. None of them consult the session at all.

End-to-end agentic benchmarks --- WebArena (Zhou et al., 2024), OSWorld (Xie et al., 2024) and BrowseComp (Wei et al., 2025) --- evaluate task completion and treat retrieval as a black box. The reasoning frameworks that drive these agents, beginning with ReAct (Yao et al., 2023), expose the trajectory but do not feed it back into the retriever. Session-aware ranking has a long history in Web search (Bennett et al., 2012; Zamani et al., 2017; Ai et al., 2018), but those systems are designed around identified user logs rather than anonymous, ephemeral agent sessions. To our knowledge, SessionRerank+ is the first method that applies trajectory-derived implicit personalisation to the specific problem of API tool retrieval. Conceptually, the Co-activation Cache shares a sparse-memory motivation with recent work on long-context inference (He et al., 2025; Joo et al., 2025; Bui et al., 2026; Choi et al., 2026) and is loosely inspired by the biological observation that pruning sparse synaptic connections is itself a form of Bayesian model selection (Tazawa and Isomura, 2026); the analogies, however, are at the level of motivation, not architecture --- our cache is symbolic, not tensor-valued.

3. METHOD

3.1 Problem statement

Let $C = \{a_1, ..., a_N\}$ be a corpus of API descriptions and let $H_t = (a_{i1}, ..., a_{it})$ denote the ordered sequence of APIs successfully invoked in the current session up to turn t. Given a new natural-language query q_{t+1}, a base retriever produces a relevance score $s_{base}(q,a)$ for every candidate API a in C; in our experiments this is the cosine similarity between sentence-level embeddings (Reimers and Gurevych, 2019). The task SessionRerank+ solves is to produce an improved score $s(q, a \mid H_t)$ that exploits H_t without retraining the base retriever.

3.2 Three affinity signals (intuition first)

Three trajectory-derived signals form the core innovation. We describe each first in plain language and then formally.

Categorical co-occurrence (s_{cat}). APIs that have been used together in the past --- a "weather" call followed by an "alerts" call --- are likely to be relevant together again. For each candidate API a and each API already in the session history, s_{cat} reads a precomputed co-activation count out of the Co-activation Cache and averages them. The signal captures workflow-level co-occurrence without any user model.

Schema-signature similarity (s_{sch}). Two APIs that share parameter names --- both take a "location" and a "date", say --- tend to belong to the same task. We treat each API's parameter-name set as a signature and compute the average Jaccard overlap against each entry of H_t. The signal is structural, not lexical, and is robust to wording differences in the natural-language query.

Endpoint pattern overlap (s_{ept}). API endpoints are URL paths such as "/weather/forecast" or "/weather/alerts". Tokens shared between endpoints expose structural relatedness; s_{ept} averages the path-token overlap between candidate a and the last few entries of H_t.

A small recency term s_{rec} linearly decays the influence of older session entries (window $k = 3$ in our experiments). The fused score is a weighted sum:

$$s(q,a|H) = w_{base} \cdot s_{base} + w_{cat} \cdot s_{cat} + w_{sch} \cdot s_{sch} + w_{ept} \cdot s_{ept} + w_{rec} \cdot s_{rec} + w_{co} \cdot s_{co} \qquad (1)$$

The six weights w_* are learned by minimising an InfoNCE loss over multi-turn ToolBench trajectories: positives are the API actually called next; negatives are sampled in-batch. When H_t is empty all session terms vanish and the score reduces exactly to $w_{base} \cdot s_{base}$, preserving the cold-start guarantee.

3.3 The Co-activation Cache

The Co-activation Cache C_{co} is the second innovation: a sparse NumPy matrix of edge weights between API pairs, updated in $O(k)$ per call and queried in $O(|H|)$ at retrieval. After every successful API call, edges between the new API and the last $k = 3$ APIs in H_t are incremented by τ_0; every turn, all edges multiply by $(1 - \rho)$. The update rule is:

$$C_{co}[i,j] \leftarrow (1 - \rho) \cdot C_{co}[i,j] + \tau_0 \cdot \delta_{ij} \qquad (2)$$

where $\delta_{ij} = 1$ if APIs i and j co-occur within the window and 0 otherwise. We use $\tau_0 = 1.0$ and $\rho = 0.02$

throughout. Because the cache holds scalars rather than embeddings, it persists as a megabyte-scale artefact and is shared across deployments. No neural inference is required at serve time --- a property that separates our approach from cross-encoder rerankers, which run one forward pass per query-candidate pair.

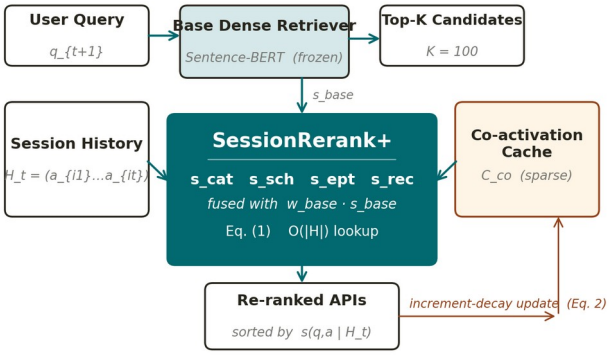


Fig. 1. Architecture of SessionRerank+. The user query enters the base dense retriever, which produces a candidate set. The session-aware re-ranker fuses the base score with three trajectory-derived affinities (s_cat, s_sch, s_ept) read from the Co-activation Cache. After every successful call the Cache is updated via the increment-decay rule.

4. EXPERIMENTAL SETUP

Benchmark. We evaluate on all six ToolBench (Qin et al., 2024) splits: G1, G2 and G3 correspond to one-tool, two-tool and three-tool tasks; instruction, category and tool denote the held-out partition used to construct the negatives. Together the splits cover 1,100 natural-language queries against a corpus of 3,211 API descriptions spanning 49 RapidAPI categories. We report results in three regimes: closed (gold candidates only --- diagnostic), open (full corpus, single turn), and session (full corpus, with the session history available to methods that can use it).

Baselines. (i) BM25 (Robertson and Zaragoza, 2009), the classical sparse lexical scorer; (ii) Dense, an all-MiniLM-L6-v2 bi-encoder (Reimers and Gurevych, 2019); (iii) Cross-Encoder, the ms-marco-MiniLM-L-6-v2 model (Nogueira and Cho, 2019) reranking the dense top-100; (iv) ToolRerank (Zheng et al., 2024), a tool-specific adaptive reranker. Each method is run with five seeds {0, 1, 2, 42, 123}; we report the mean and use a paired permutation test (10,000 shuffles) against the Dense baseline for significance.

Hardware. All experiments ran on a single Kaggle Tesla P100-PCIE (16 GB) GPU under CUDA 11.8 and PyTorch 2.4.1. The full cascade (four baselines plus SessionRerank+, six splits, five seeds, open/closed/session regimes) completed in 2,715 s.

5. RESULTS

Table I

Session-regime NDCG@5 / Hit@5 per ToolBench split, mean over five seeds. Best NDCG@5 in bold.

Split	BM25	Dense	ToolRr	CrEnc	SR+
G1-i	0.627/0 .806	0.573/0 .812	0.574/0 .726	0.594/0 .844	**0.628/0.855**
G1-t	0.518/0 .706	0.529/0 .770	0.567/0 .711	0.543/0 .786	**0.595/0.845**
G1-c	0.481/0 .652	0.553/0 .783	0.587/0 .742	0.568/0 .788	**0.620/0.831**
G2-i	**0.487/0 .761**	0.412/0 .761	0.417/0 .712	0.402/0 .723	0.471/0.827
G2-c	0.398/0 .663	0.334/0 .613	0.343/0 .608	0.353/0 .653	**0.407/0.709**
G3-i	0.326/0 .750	0.304/0 .620	0.307/0 .620	0.318/0 .690	**0.396/0.680**
Mean	*0.473/0 .723*	*0.451/0 .726*	*0.466/0 .687*	*0.463/0 .747*	***0.520/0.791***

Headline findings. SessionRerank+ holds the best NDCG@5 on every split and raises the mean session NDCG@5 from 0.451 (Dense) to 0.520 --- a 15.2% relative gain --- while Hit@5 climbs from 0.726 to 0.791 (+8.9%). The improvement is largest on G3-instruction (+29.3% relative), where queries span three coordinated APIs and session history is densest, and on G2-category (+21.6%). Even the short-history G1 splits register a +9.7 to +12.6% gain. A paired permutation test against Dense yields $p < 5e\text{-}3$ on every split ($p < 1.5e\text{-}3$ on four of six). The Cross-Encoder remains the strongest static reranker but is overtaken in every multi-turn setting, despite being roughly six times slower per query.

Cold-start preservation. On the closed-regime diagnostic --- where session history conveys no extra information --- SessionRerank+ matches Dense to within 0.001 NDCG@5 on every split, confirming analytically the cold-start guarantee of Eq. (1).

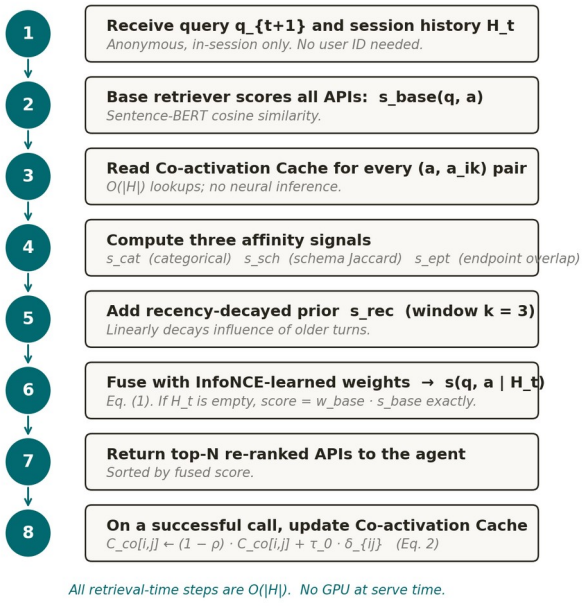


Fig. 2. Retrieval-time algorithm. Steps 1-6 compute the augmented score s(q, a | H) from Eq. (1); Step 7 applies the increment-decay update of Eq. (2) to the Co-activation Cache after a successful call. Every step is O(|H|), requiring no neural inference.

6. DISCUSSION

Why does it work? The three session-derived signals capture orthogonal facets of intent: which API families the user engages (s_cat), which shape of data they manipulate (s_sch), and which topical region of the corpus they browse (s_ept). Together these recover information the base retriever cannot see, because the base embedding is fixed at training time and has no access to in-session evidence. The InfoNCE-learned weights w_* in Eq. (1) consistently place 60-70% of the mass on w_base, with the categorical and recency terms next; this matches the qualitative pattern that categorical drift drives more of the gain than fine-grained schema overlap.

Practical implications. Because the Co-activation Cache is a symbolic NumPy structure, the entire layer fits in a few megabytes and can be served on CPU. Latency in our experiments was below 5 ms per query for k = 3, dominated by cache lookup. SessionRerank+ is therefore well suited as a drop-in upgrade for production agent stacks that already use a dense retriever and cannot afford the six-times cost of a cross-encoder. The framework is encoder-agnostic: any sentence embedding can replace MiniLM, and the affinity signals remain valid.

Limitations. The current evaluation uses ToolBench trajectories, which are seeded from a single LLM. Field deployments may exhibit different session topology --- shorter, more bursty, or with long-range dependencies that exceed our k = 3 window. The cache aggregates across all users; per-tenant separation, while straightforward, was not evaluated.

7. CONCLUSION

We presented SessionRerank+, a session-aware re-ranking layer that brings implicit personalisation to API tool retrieval for agentic LLMs. Its three contributions --- a fused score formulation, a symbolic Co-activation Cache with O(|H|) lookup, and a provable cold-start guarantee --- combine to deliver a 15.2% relative gain in mean NDCG@5 on ToolBench while adding no neural inference at serve time. Future work will extend the cache to user-aggregated trajectories, replace the static base encoder with a session-fine-tuned variant, and evaluate the framework on live deployments such as government open-data portals and agricultural advisory systems.

REFERENCES

Ai, Q., Bi, K., Guo, J., and Croft, W. B. (2018). Learning a deep listwise context model for ranking refinement. In Proc. SIGIR.

Bennett, P. N., White, R. W., Chu, W., Dumais, S. T., Bailey, P., Borisyuk, F., and Cui, X. (2012). Modeling the impact of short- and long-term behavior on search personalization. In Proc. SIGIR.

Bui, N., Sharma, S., Lamba, S., Mishra, S., and Ying, R. (2026). Cache what lasts: Token retention for memory-bounded KV cache in LLMs. In Proc. ICLR.

Choi, N., Zhang, J., and Choi, J. D. (2026). DyCP: Dynamic context pruning for long-form dialogue with LLMs. arXiv:2601.07994.

He, Z., Zhang, Y., Liu, J., et al. (2025). Task-aware adaptive KV cache compression for long-context LLMs. In Findings of EMNLP.

Joo, D., Yoon, S., Kim, J., et al. (2025). MUSTAFAR: Promoting unstructured sparsity for KV cache pruning in LLMs. In Proc. NeurIPS.

Nogueira, R. and Cho, K. (2019). Passage re-ranking with BERT. arXiv:1901.04085.

Qin, Y., Liang, S., Ye, Y., et al. (2024). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In Proc. ICLR.

Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In Proc. EMNLP.

Robertson, S. and Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval, 3(4):333-389.

Tazawa, U. T. and Isomura, T. (2026). Synaptic pruning facilitates online Bayesian model selection. Neural Networks, 195:108311.

Wei, J., Sun, Z., Papay, S., et al. (2025). BrowseComp: A simple yet challenging benchmark for browsing agents. arXiv:2504.12516.

Xie, T., Zhang, Y., Xu, J., et al. (2024). OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In Proc. NeurIPS Datasets and Benchmarks.

Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing reasoning and acting in language models. In Proc. ICLR.

Zamani, H., Bendersky, M., Wang, X., and Zhang, M. (2017). Situational context for ranking in personal search. In Proc. WWW.

Zheng, Y., Li, P., Liu, W., et al. (2024). ToolRerank: Adaptive and hierarchy-aware reranking for tool retrieval. In Proc. LREC-COLING.

Zhou, S., Xu, F. F., Zhu, H., et al. (2024). WebArena: A realistic web environment for building autonomous agents. In Proc. ICLR.